Universal Object Reference Framework: A Prism Implementation for High-Dimensional Semantic Preservation and Lossy Compression

The exponential acceleration of semantic web applications, agentic artificial intelligence, and large language models has fundamentally altered the landscape of data management. Contemporary computational architectures increasingly rely on high-dimensional vector embeddings to capture the intended notions of semantic similarity as geometric distances. Typically ranging from 100 to 1,000 dimensions, these embeddings are essential for translating complex modalities—such as natural language, imagery, and autonomous telemetry—into machine-readable spatial coordinates. However, manipulating these expansive topological spaces introduces severe computational bottlenecks, widely referred to within the domain as the "curse of dimensionality". As dimensionality scales, the computational cost of calculating vector distances rises disproportionately, necessitating the implementation of lossy data compression techniques to make retrieval and storage economically viable.

While traditional dimensionality reduction and vector quantization algorithms effectively reduce memory footprints, they suffer from a fatal flaw: the destruction of intrinsic semantic topology. The heuristic clustering of high-dimensional continuous vectors into discrete centroids frequently creates a "semantic gap," where the resulting vector representation diverges entirely from the true attributes of the original entity. Furthermore, attempting to compress multi-dimensional tensors by arbitrarily flattening them into one-dimensional arrays significantly degrades compression ratios—often by factors between 1.2\times and 1.3\times—while imposing algorithmic performance penalties exceeding 17.5\% per operation.

To structurally resolve this paradox, a paradigm shift is required—moving away from statistical approximation and toward mathematically invariant signal processing. The Universal Object Reference (UOR) Framework provides this exact unified computational substrate. Functioning as a shared, universal coordinate system for information, the UOR Framework guarantees persistent identity, trustless verification, and universal composability. By formalizing a mathematical architecture for content-addressed, symmetric, and multi-metric object spaces based on the algebraic structure of Z/(2^n)Z, the framework completely isolates semantic meaning from observer bias.

This report outlines the synthesis of a UOR-compatible "Prism" implementation. Engineered as an immediate, deployable use case for high-dimensional semantic routing, the Prism leverages the UOR taxonomy and the mathematical invariants of the Atlas of Resonance Classes. Through advanced Spectral Prime Decomposition, the implementation facilitates extreme lossy compression of geometric data while preserving the absolute semantic truth of the underlying entities, thereby resolving the high-dimensional compression crisis for agentic AI.

The Architecture of the Universal Object Reference

Framework

The UOR Framework completely fundamentally redesigns how digital objects are identified, stored, and verified in distributed systems. Rather than relying on fragile, location-based URIs or model-dependent vector indices, the framework establishes a rigorous, cryptographically stable identity system.

Content-Derived Addressing and Verification

At the foundation of the UOR architecture is the principle of content-derived addressing. Every object within the universal coordinate system is assigned exactly one address that is mathematically derived directly from its intrinsic content. This ensures a persistent identity across disparate systems, eliminating issues related to opaque coordination, identity fraud, and broken links.

The trustless verification of this identity does not rely on a centralized certificate authority or consensus mechanism. Instead, it relies on a structural "critical identity" grounded in the framework's Z/(2^n)Z ring mathematics. The core invariant equation utilized for verification is: For any data entity x mapped into the UOR object space, the negation of its bitwise NOT must be mathematically equivalent to its successor. Empirical validation is straightforward; substituting x = 42 into the equation yields neg(bnot(42)) = neg(213) = 43, which equates exactly to succ(42).

This algebraic identity serves as a continuous, structural checksum for all objects in the universal coordinate system. The framework exposes a live API (operating at https://api.uor.foundation/v1) that allows agents to query this property instantly without authentication.

API Endpoint	Core Functionality within the UOR Kernel
GET /navigate	Provides a complete hierarchical index of the 48 available OpenAPI 3.1 endpoints.
GET /kernel/op/verify	Executes the critical identity verification, returning proof witnesses for the targeted object.
GET /kernel/op/compute	Performs all fundamental ring operations on a provided variable to establish coordinate context.
GET /bridge/trace	Computes mathematical transformations to detect "Hamming drift," thereby identifying prompt injections or data corruption.

Through the /bridge/trace endpoint, the UOR Framework actively monitors for "Hamming drift". In high-dimensional vector databases, malicious prompt injections or data degradation can subtly shift floating-point coordinates. Because the UOR Framework relies on strict ring invariants, any unauthorized coordinate manipulation alters the bitwise topology, breaking the neg(bnot(x)) = succ(x) identity. This immediately flags the object as compromised, providing unprecedented structural security for LLM context windows.

Trilateral Coherence: Decoupling the Observer from the Object

The fundamental flaw in modern vector embeddings is the conflation of the object's meaning

with the neural network's perspective. If the embedding model (e.g., a specific iteration of a large language model) changes, the spatial coordinates of the vector change, necessitating the complete recalculation and garbage collection of the index.
To solve this, the UOR Framework introduces the concept of Trilateral Coherence, which maintains strict consistency between three decoupled nodes of existence :

1. **Objects:** The fundamental, invariant entities that exist independently of any measurement.
2. *Representations:* The base-independent, canonical mathematical forms of these objects.
3. *Observer Frames:* The specific, relativistic perspectives, coordinate systems, or embedding models from which the objects are viewed.

Trilateral Coherence acts as a geometric governor. It ensures that while an object's coordinates may shift depending on the specific Observer Frame (the specific AI model's embedding space), the underlying Canonical Representation remains universally identical. Transform functions dictate the mapping between observer frames, preserving essential information during coordinate translation. Coherence measures actively quantify this representational integrity during all write operations, leveraging the kernel invariants inherited directly from the UOR abstract classes.
By completely isolating the observer's bias from the intrinsic canonical representation, Trilateral Coherence eradicates the "semantic gap".

Rigorous Taxonomy and the Version 0.1.1 Ontology

A universal coordinate system is only functional if it possesses a rigidly defined, universally accessible taxonomy. The UOR Foundation ontology provides the strict classification schemas required to map highly complex semantic data into mathematical forms without losing context during compression.

Structural Scope of the Taxonomy

As of Version 0.1.1, the UOR ontology represents an exhaustive mapping of semantic and structural boundaries. The taxonomy is meticulously organized into the following scale:

- **23 Namespaces:** These namespaces partition different semantic domains, allowing for optimized cross-namespace queries and decentralized content management.
- **310 Classes:** Classes define the rigid schemas for objects, observer frames, and computational nodes within the coordinate system.
- **704 Properties:** The properties define the specific directional edges, attributes, and mathematical relationships between classes.
- **1,292 Named Individuals:** These individuals serve as permanent, fixed anchor points within the object space, enabling absolute coordinate calibration across distributed networks.

This exhaustive taxonomy ensures that every conceivable dimension of a semantic vector can be accurately tagged, categorized, and preserved. The source of truth for all taxonomy terms is maintained as statically typed Rust data within the spec/ directory (specifically within the uor-ontology crate) of the UOR-Framework repository.

Export Formats and Interoperability

To ensure immediate use case applicability across the diverse ecosystem of semantic web applications and agentic AI, the typed Rust data is aggressively serialized into numerous standardized export formats. The framework's internal code generation logic (uor-codegen) automatically compiles the ontology into the uor-foundation crate, providing typed Rust traits that are published to crates.io. This allows system architects to natively integrate UOR taxonomy checking at compile-time in Rust environments, maximizing runtime performance during high-speed vector routing.
For broader system interoperability, the taxonomy is concurrently exported as:

- **JSON-LD 1.1** (public/uor.foundation.jsonld)
- **Turtle 1.1** (public/uor.foundation.ttl)
- **N-Triples** (public/uor.foundation.nt)
- **OWL 2 RDF/XML** (public/uor.foundation.o[span_50](start_span)[span_50](end_span)wl)

Crucially for data validation during lossy compression pipelines, the framework provides a **JSON Schema (Draft 2020-12)** and **SHACL validation shapes**. These artifacts mathematically guarantee that compressed semantic payloads do not violate the boundaries of the 310 established classes. An accompanying **EBNF grammar** provides formal parsing rules for all taxonomic elements, ensuring that cross-system deserialization is flawlessly executed.

Hardware Integration and Storage Considerations

The taxonomy also accounts for the physical realities of computational hardware, ensuring that mathematical abstractions do not cause memory fragmentation. The framework categorizes computational environments based on their natural boundaries, differentiating between "byte-storage computers" (which lack natural boundaries other than the byte) and "word-storage computers" (which operate on multiples of the fundamental byte boundary).
When allocating numeric storage for universal object references, the taxonomy includes constraints akin to legacy memory allocation (such as the SYNCHRONIZED clause principles in mainframe languages), ensuring that representations occupy the minimum necessary bytes without triggering memory alignment penalties. Furthermore, backward-compatibility paradigms are maintained regarding universal and typed object references, ensuring that methods referencing self-instantiated objects (e.g., binding an object to SELF) resolve predictably across different observer hardware architectures.

Mathematical Foundations of the Prism Implementation

To solve the lossy compression crisis within high-dimensional spaces, the architecture deploys the "Prism" implementation. Conceptually akin to an optical prism that splits light into fundamental, invariant frequencies without destroying the energy, the UOR Prism mathematically decomposes dense floating-point vectors into their irreducible algebraic structures.

The Atlas of Resonance Classes

The theoretical bedrock of the Prism's mapping capabilities is the "Atlas of Resonance Classes." Traditional vector databases map embeddings into an open-ended Euclidean space, leading to arbitrary clustering. Conversely, the Prism projects continuous vectors onto a highly structured,

bounded topology.
The Atlas is mathematically defined as a 96-vertex graph that manifests dynamically as the stationary configuration of a discrete action functional acting upon a 12,288-cell boundary complex. The action functional is elegantly defined as:
Where the action S[\phi] is the summation over the boundary cells. It is imperative to note that the Atlas is not generated algorithmically through trial and error; it is the absolute, unique configuration satisfying this functional.

Topological Property	Description and Function within the Prism
96 Vertices	Define the absolute Resonance Classes, explicitly labeled by E_8 coordinates, linking the topology to exceptional Lie groups.
Mirror Symmetry (\tau)	Provides a canonical involution across the space, ensuring that distance calculations remain symmetric across transformations.
12,288-Cell Boundary	Establishes the discrete geometric domain upon which the action functional operates, forming a closed coordinate system.
Unity Constraint	Governs the adjacency of nodes, ensuring that vector proximity is strictly determined by roots of unity rather than heuristic floating-point proximity.

Recent breakthroughs in spectral analysis have verified the Laplacian spectrum of this precise 96-vertex graph using exact rational arithmetic—eliminating floating-point approximations entirely. The analysis definitively proved that the spectral gap (the smallest non-zero eigenvalue) is exactly \lambda_1 = 1.
In the context of high-dimensional semantic clustering, a spectral gap of exactly 1 implies optimal expander graph properties. The graph is simultaneously highly connected and remarkably sparse. When a 1,000-dimensional continuous vector is projected onto this discrete 12,288-cell complex, the projection instantly resolves to the nearest invariant resonance classes without the cascading error propagation common in k-means vector quantization.

Signal Processing and Spectral Prime Decomposition

With the geometric space constrained by the Atlas of Resonance Classes, the Prism executes data compression not by discarding data statistically, but by factorizing it algebraically. This process is termed Spectral Prime Decomposition.
Instead of assuming language and semantics are mere token-sequence probabilities, the Prism treats high-dimensional vector embeddings as complex waveforms or signals. Within this paradigm, the embedding vector contains the "true" semantic signal, heavily obscured by the "noise" of the embedding model's observer bias.
The decomposition process extracts this signal by factorizing the object into its irreducible mathematical components. The factorization is mathematically guaranteed to be both unique and entirely observer-independent. To achieve this computationally, the Prism utilizes coherence-gradient descent algorithms. The algorithm descends the topological gradient of the embedding, isolating the primary associated primes of the object within the Z/(2^n)Z polynomial ring structures.
By identifying the minimal associated primes and extracting their primary components, the

algorithm yields a discrete, finite set of prime factors. Because every integer (and by extension, every digitized semantic object in this coordinate system) possesses a unique prime factorization, the semantic meaning is preserved flawlessly.

Base-Independent Canonical Representation

The extracted prime factors must be stored in a manner that transcends the limitations of localized hardware architecture. The Prism implementation achieves this by converting the decomposed factors into "Universal Numbers" using the @uor-foundation/math-js Prime Math Library.

A Universal Number is a fundamental integer encoded by its complete digit expansions across all mathematical bases. This base-independent representation guarantees that whether the data is analyzed in base-2, base-10, or base-16, the intrinsic prime factor structure remains geometrically consistent.

When the Prism algorithm completes, the massive, dense floating-point vector is replaced by a Canonical Representation constructed of these Universal Numbers. This canonical form is drastically smaller in memory footprint than the original vector—achieving the goals of lossy compression regarding overall file size—but the compression is entirely lossless regarding the semantic topology. The redundant observer bias has been discarded, while the irreducible meaning is retained permanently within the UOR framework invariants.

Architecting the Immediate Use Case: The Model Context Protocol (MCP) Implementation

For the Prism technology to serve as an immediate use case, it must be rapidly deployable into existing AI ecosystems. The implementation satisfies this requirement by surfacing the UOR core features—Prime Decomposition, Canonical Representation, and Trilateral Coherence—through a standardized Model Context Protocol (MCP) server.

Decoupled Storage and Decentralized Architecture

Unlike legacy vector databases that require centralized infrastructure, massive RAM allocations, and continuous garbage collection routines , the MCP UOR implementation embraces a fully decentralized topology.

The implementation utilizes GitHub integration for the persistent storage, version control, and authentication of UOR data. Because Canonical Representations are highly compressed arrays of Universal Numbers, they can be efficiently version-controlled via standard Git protocols. When an object is decomposed and its semantic meaning extracted, the resulting Canonical Representation is committed to a GitHub repository, tagging the specific 23 namespaces and 310 classes dictated by the UOR taxonomy.

To ensure seamless integration for end-user applications (such as custom LLM frontends or agentic orchestration platforms), the architecture employs a Netlify deployment mechanism. Authentication is handled via a built-in Netlify OAuth proxy. The integration strictly defines the OAuth callback URL (https://api[span_84](start_span)[span_84](end_span).netlify.com/auth/done), securely bridging the AI agent's environment with the GitHub-hosted decentralized semantic storage.

Operational Workflow of the MCP Server

When a Large Language Model interfaces with the UOR MCP Server, it gains access to a standardized suite of operational tools designed to interact with the universal coordinate system. The workflow operates as follows:

1. **Tool Discovery (tools/list):** The LLM queries the server to retrieve the available UOR operations. This includes tools for submitting vectors for decomposition, querying cross-namespace data, and verifying object integrity.
2. **Resource Discovery (resources/list):** The LLM can retrieve a list of available UOR resources, essentially scanning the GitHub-hosted data repository to find pre-computed Canonical Representations.
3. **Execution (tools/call):** The LLM executes a specific UOR operation. For example, if the LLM needs to understand the semantic correlation between two high-dimensional concepts, it calls the MCP server.

When a new high-dimensional vector is submitted for compression, the MCP server invokes the Prism logic. The vector is tagged with its Observer Frame (e.g., "OpenAI-ada-002" or "Cohere-embed-v3"). The Spectral Prime Decomposition algorithm strips the observer noise, mapping the data onto the 12,288-cell complex and extracting the E_8 resonance classes. Crucially, the decomposition is cached for extreme performance optimization but is designed to be recalculated dynamically if the underlying semantic objects change, completely eliminating the stale data issues that plague traditional vector garbage collection.

Cross-Namespace Resolution and Universal Composability

A defining triumph of the MCP Prism implementation is its capacity for cross-namespace queries. In legacy systems, data formatted for financial risk assessment is geometrically incompatible with data formatted for biological taxonomy.

However, because the UOR Framework forces all objects to resolve to base-independent Canonical Representations governed by the exact same 704 properties and 23 namespaces, data becomes universally composable. The MCP server's built-in namespace resolution logic allows an AI agent to cross-reference a biological dataset seamlessly with an economic dataset, calculating absolute semantic similarity based purely on the shared prime factors of their Universal Numbers.

This guarantees that the structural truth of the data is mathematically maintained, irrespective of the external observer or the domain in which it originated.

Workspace Mechanics and Conformance Validation

The success of the Prism use case is heavily dependent on the rigorous software engineering standards maintained within the UOR-Foundation repository. The repository structure is meticulously crafted to enforce the mathematical invariants of the taxonomy at the compiler level.

Repository Crate Architecture

The Rust workspace is segmented into distinct crates, each managing a critical lifecycle phase of the universal coordinate system :

- **uor-ontology (spec/):** The immutable source of truth. Any changes to the semantic taxonomy (e.g., adding a new namespace or class) must originate here.
- **uor-codegen (codegen/):** A meta-programming crate that reads the static typed Rust data in the spec/ directory and automatically generates the highly optimized Rust traits used for mathematical operations.
- **[span_56](start_span)[span_56](end_span)uor-foundation (foundation/):** The output of the codegen crate. This is the actual library that is published to crates.io and imported by the MCP server and external applications to interface with the UOR system. * **uor-conformance (conformance/):** A vital crate that houses workspace-wide conformance validators. It mathematically guarantees that any generated Universal Number or SHACL validation shape strictly adheres to the Z[span_58](start_span)[span_58](end_span)/(2^n)Z algebraic structure and the neg(bnot(x)) = succ(x) critical identity.

Continuous Integration and Deployment Tooling

To orchestrate these crates, the framework includes a suite of client binaries within the clients/ directory (uor-clients). These binaries automate the entire build and validation process:

- cargo run --bin uor-build synthesizes the raw ontology artifacts.
- cargo run --bin uor-crate triggers the code generation for the Rust traits.
- cargo ru[span_59](start_span)[span_59](end_span)n --bin uor-docs and uor-website generate the exhaustive documentation and static web assets.
- cargo run --bin uor-conformance executes the entire suite of mathematical validators.

The Continuous Integration (CI) pipeline further enforces strict quality control by executing cargo fmt --check for structural consistency, cargo clippy -- -D warnings to treat all linting warnings as hard compiler errors, and cargo test to execute unit tests against the prime decomposition algorithms. This unyielding software engineering rigor ensures that the UOR Framework remains an entirely trustless, mathematically pristine environment for high-dimensional operations.

Solving the Crisis of Dimensionality: Comparative Analysis

To contextualize the transformative nature of the UOR Prism implementation, it is necessary to conduct a direct comparative analysis against prevailing high-dimensional compression methodologies.

The Failure of Flattening and Principal Component Analysis

When dealing with massive multidimensional arrays, system architects often attempt naive dimensionality reduction by flattening the tensors into 1D arrays or applying Principal Component Analysis (PCA). Empirical measurements reveal that treating multidimensional data buffers as 1D arrays immediately degrades the compression ratio by up to 1.3\times.
More alarmingly, these techniques assume that variance is equivalent to semantic importance. By discarding axes with lower variance, PCA irreversibly destroys the intricate topological features of the data. While this may be acceptable for rendering a low-resolution image, in the context of agentic AI and semantic web routing, destroying topological features fundamentally

alters the logical reasoning the AI can derive from the data.
The Vector Quantization Trap
Modern vector databases attempt to solve the storage crisis through Vector Quantization (VQ). VQ operates by partitioning the continuous R^D space into a finite number of regions (Voronoi cells), each represented by a discrete centroid. When a new vector is ingested, it is "snapped" to the nearest centroid.
While this achieves aggressive lossy compression, it is geometrically disastrous for semantic preservation. The clustering algorithms operate solely on distance metrics (e.g., cosine similarity), entirely blind to the actual meaning of the data. If two semantically distinct concepts happen to fall near the boundary of the same Voronoi cell, they will be compressed into the identical centroid, permanently erasing the distinction between them—a textbook manifestation of the semantic gap. Furthermore, VQ requires massive computational overhead for index construction and continuous garbage collection to purge obsolete indices as the dataset evolves.

The Prism Advantage: Mathematical Invariance

The UOR Prism bypasses these pitfalls entirely by shifting the compression paradigm from geometric clustering to algebraic factorization.

Feature	Legacy Vector Quantization (VQ)	UOR Prism Implementation
Data Representation	Approximate, floating-point coordinates assigned to heuristic centroids.	Exact, base-independent Universal Numbers derived from prime factorization.
Semantic Fidelity	Highly lossy; topological relationships are routinely destroyed by boundary collisions.	Structurally lossless; compression strips observer noise but preserves exact irreducible signal invariants.
Topology	Infinite, unbounded Euclidean space (R^D) subject to the curse of dimensionality.	Discrete, bounded 12,288-cell boundary complex mapped to the Atlas of Resonance Classes.
Observer Independence	Completely dependent on the embedding model; changing the model requires full database recreation.	Trilateral Coherence ensures Canonical Representations are universally independent of the Observer Frame.
Security	Vulnerable to prompt injections and subtle coordinate manipulation (data poisoning).	Cryptographically secured by the critical identity neg(bnot(x)) = succ(x) and continuous Hamming drift tracing.

By utilizing the 96-vertex Atlas of Resonance Classes, the Prism provides a finite, rigidly structured topology. When a high-dimensional vector is decomposed, it is not "snapped" to an arbitrary statistical centroid; it is mathematically reduced to its foundational E_8 coordinates via coherence-gradient descent.
Because the resulting Universal Number is an exact algebraic reality, the semantic meaning is preserved with zero degradation. The "loss" in this lossy compression strictly applies to the redundant, noisy dimensions introduced by the specific neural network embedding (the

Observer Frame). The core semantic signal remains utterly intact, immediately verifiable, and universally composable across any system utilizing the UOR ontology.

Future Outlook: The Signal-Based Semantic Web

The deployment of the UOR Prism implementation signifies a fundamental transition in how artificial intelligence interfaces with human knowledge. Traditional Large Language Models operate under the assumption that statistical patterns within token sequences can adequately approximate semantic understanding. This token-based, probabilistic approach has plateaued, yielding systems plagued by hallucination, incoherence, and insurmountable context window limitations.

The integration of the Universal Object Reference Framework treats language understanding and multi-modal semantics as rigorous signal processing problems. By establishing a universal coordinate system where digital entities are assigned mathematically verifiable, content-derived addresses, the framework ensures that data acts as an objective, structural reality rather than a statistical approximation.

Accelerating Frontier Technologies

The implications of this implementation extend far beyond standard AI text generation. The strict Trilateral Coherence and the robust 23-namespace taxonomy create an environment ideally suited for frontier scientific research.

In fields requiring massive data coordination—such as climatology, genomic sequencing, and autonomous drone telemetry—the UOR Prism allows distributed nodes to share incredibly dense, high-dimensional conceptual data across low-bandwidth networks. Because the data has been compressed to its minimal prime factors, it can be transmitted instantaneously. Upon receipt, the mathematical invariants guarantee that the receiving node comprehends the exact topological structure of the data, completely eliminating the translation errors that historically derail open-science initiatives.

Eradicating the Hallucination Paradigm

Furthermore, the structural integrity provided by the Z/(2^n)Z ring mathematics acts as an absolute defense mechanism. The integration of the /bridge/trace endpoint for real-time Hamming drift detection means that autonomous agents can finally operate in a truly trustless environment. If an adversarial node attempts to inject malicious logic into a high-dimensional semantic vector, the vector's prime factorization will fundamentally alter, causing the neg(bnot(x)) = succ(x) identity check to fail. The compromised data is instantly rejected by the coordinate system, structurally preventing AI hallucinations and opaque coordination failures.

Synthesis

The Universal Object Reference (UOR) Framework has delivered a mathematical architecture capable of supporting the next generation of artificial intelligence. The 0.1.1 ontology specification provides a sprawling, 310-class taxonomy exported across multiple robust formats (including JSON-LD, SHACL, and Rust traits), ensuring that no metadata is lost during cross-system communication.

By synthesizing this framework into the Prism implementation, the paradox of high-dimensional lossy compression is unequivocally resolved. The integration of Trilateral Coherence successfully isolates arbitrary observer bias from the intrinsic canonical representations of objects. Concurrently, the application of Spectral Prime Decomposition across the 12,288-cell Atlas of Resonance Classes replaces destructive vector quantization with exact algebraic factorization.

The resulting base-independent Universal Numbers provide extreme data compression while perfectly preserving the semantic topology of the original embedding. Deployed as an immediate use case via the Model Context Protocol and decentralized GitHub storage, the UOR Prism empowers AI agents to navigate, manipulate, and coordinate across high-dimensional semantic spaces with absolute mathematical certainty, effectively unlocking the true potential of the semantic web.

Works cited

1. Data Engineering - IEEE Computer Society, http://sites.computer.org/debull/A24sept/A24SEPT-CD.pdf 2. What Is Vector Search? The Ultimate Guide - Oracle, https://www.oracle.com/database/vector-search/ 3. Productive and Performant Generic Lossy Data Compression with LibPressio - NSF Public Access Repository, https://par.nsf.gov/servlets/purl/10319848 4. FFT & Wiener Filters - GitHub Gist, https://gist.github.com/usrbinkat/e40b4d643bba6ff06ca63d6a5ef06aa1 5. The UOR Foundation — Your Universal Coordinate System for Information, https://uor.foundation/ 6. UOR-Foundation/UOR-Framework: Formalization of the ... - GitHub, https://github.com/UOR-Foundation/UOR-Framework 7. UOR Framework MCP Server – README - UBOS.tech, https://ubos.tech/mcp/model-context-protocol-mcp-uor-implementation/ 8. UOR Framework MCP Server – Overview | MCP Marketplace - UBOS, https://ubos.tech/mcp/model-context-protocol-mcp-uor-implementation/overview/ 9. Concepts of the COBOL Language, https://www.microfocus.com/documentation/reuze/60d/lhclan.htm 10. COBOL For AIX Programming Guide-Igyapg00 | PDF - Scribd, https://www.scribd.com/document/727094158/COBOL-for-AIX-Programming-Guide-igyapg00 11. atlas_embeddings - Rust - Docs.rs, https://docs.rs/atlas-embeddings 12. Paraxiom/resonance-spectral-gap: Exact proof - GitHub, https://github.com/Paraxiom/resonance-spectral-gap 13. afflom's gists · GitHub, https://gist.github.com/afflom?direction=asc&sort=created 14. an algorithm to compute a primary decomposition of modules in polynomial rings over the integers - AKJournals, https://akjournals.com/downloadpdf/view/journals/012/52/1/article-p40.pdf 15. Reconstruct an integer from its prime exponents - Code Golf Stack Exchange, https://codegolf.stackexchange.com/questions/220260/reconstruct-an-integer-from-its-prime-exponents 16. Decomposition of number into prime numbers - Stack Overflow, https://stackoverflow.com/questions/7755771/decomposition-of-number-into-prime-numbers 17. uor-foundation/math-js - NPM, https://www.npmjs.com/package/@uor-foundation/math-js 18. Lossy Compression of Scientific Data: Applications Constrains and Requirements NSF FZ Project Sarasota Workshop Report - arXiv, https://arxiv.org/html/2503.20031v1 19. The UOR Foundation - GitHub, https://github.com/UOR-Foundation